



UNIDAD II · SEMANA 6

Web Services RESTful con Spring Boot

Paso a paso: del «Hola Mundo» al CRUD completo — con relaciones JPA y claves primarias de doble campo (compuestas).

Spring Boot 4.1 · Java 25 LTS · Maven · PostgreSQL 18 · IntelliJ IDEA 2026.1

La ruta: ocho pasos hacia una API REST



1 • Crear el proyecto

Spring Initializr en IntelliJ IDEA Ultimate.



3 • Arquitectura por capas

Paquetes config, model, repository, service, controller, dto.



5 • Clave compuesta

Tabla puente con PK de doble campo (@EmbeddedId).



7 • Errores y validación

@RestControllerAdvice + @Valid (400/404).



2 • Hola Mundo REST

Primer @RestController que devuelve JSON.



4 • Modelo de dominio

Entidades JPA: 1:1, 1:N, N:1 y N:M con atributos.



6 • CRUD REST

Repositorios, servicio y controlador con los cinco verbos.



8 • Documentar

OpenAPI 3 y Swagger UI con springdoc 3.

Qué construiremos: una API académica de UTEQ

Dominio del ejemplo

Carrera

agrupa muchos estudiantes (1 : N)

Estudiante

tiene un Perfil (1 : 1)

Estudiante ↔ Curso

muchos a muchos, con nota y periodo (N : M)

Inscripcion

tabla puente con PK (estudiante_id, curso_id)

Endpoints que expondremos

```
GET    /api/estudiantes
GET    /api/estudiantes/{id}
POST   /api/estudiantes
PUT    /api/estudiantes/{id}
DELETE /api/estudiantes/{id}
POST   /api/inscripciones
GET    /api/estudiantes/{id}/cursos
GET    /swagger-ui.html
```

Requisitos y versiones de referencia

Verificado a junio de 2026. Spring Boot 4 corre sobre Spring Framework 7 y Jakarta EE 11 (paquetes jakarta.).*

Herramienta	Versión	Rol en el proyecto
JDK	Java 25 LTS (o 21 LTS)	Lenguaje y máquina virtual
Spring Boot	4.1.0	Framework: auto-config, Tomcat embebido
Build tool	Maven 3.9+	Dependencias y empaquetado
Base de datos	PostgreSQL 18	Persistencia relacional
ORM	Spring Data JPA / Hibernate 7	Mapeo objeto-relacional
Documentación	springdoc-openapi 3.0.3	OpenAPI 3 + Swagger UI
IDE	IntelliJ IDEA Ultimate 2026.1	Editor, Inicializ, ejecución
Cliente HTTP	Postman / cURL	Probar los endpoints

New Project → Spring Boot (Spring Initializr)

Asistente de IntelliJ IDEA

File → **New** → **Project** → **Spring Boot**

Language: Java Type: Maven

JDK: 25 Java: 25

Packaging: Jar

Group: ec.edu.uteq

Artifact: appweb-api

Spring Boot: 4.1.0

Dependencies (starters)

- ✓ **Spring Web** — API REST con @RestController
- ✓ **Spring Data JPA** — Repositorios y ORM Hibernate
- ✓ **PostgreSQL Driver** — Conector JDBC de PostgreSQL
- ✓ **Validation** — Bean Validation (@Valid)
- ✓ **Lombok** — Getters/setters sin boilerplate
- ✓ **Spring Boot DevTools** — Recarga en caliente

springdoc-openapi 3.0.3 se añade luego, manualmente, en el pom.xml (no está en el Initializr).

La estructura que genera IntelliJ

```
appweb-api/  
├─ pom.xml ← dependencias Maven  
├─ mvnw, mvnw.cmd wrapper de Maven  
├─ src/  
│   ├── main/  
│   │   ├── java/ec/edu/uteq/appwebapi/  
│   │   │   └─ AppwebApiApplication.java main()  
│   │   └─ resources/  
│   │       ├── application.properties  
│   │       ├── static/ css, js  
│   │       └─ templates/  
│   └─ test/java/.../ pruebas
```

Clase principal

```
@SpringBootApplication  
public class  
    AppwebApiApplication {  
    public static void main(  
        String[] a) {  
        SpringApplication.run(  
            AppwebApiApplication  
                .class, a);  
        }  
    }  
}
```

Punto de arranque: levanta Tomcat embebido en el puerto 8080.

pom.xml: el corazón del proyecto Maven

pom.xml (fragmento)

```
<parent>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-parent</artifactId>
  <version>4.1.0</version>
</parent>

<dependencies>
  <dependency>    <!-- API REST -->
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-web</artifactId>
  </dependency>
  <!-- data-jpa, validation, devtools: igual patron -->
  <dependency>    <!-- driver PostgreSQL -->
    <groupId>org.postgresql</groupId>
    <artifactId>postgresql</artifactId>
  </dependency>
  <dependency>    <!-- Swagger UI / OpenAPI 3 -->
    <groupId>org.springdoc</groupId>
    <artifactId>springdoc-openapi-starter-webmvc-ui</artifactId>
    <version>3.0.3</version>
  </dependency>
</dependencies>
```

Primer @RestController que devuelve JSON

HolaController.java

```
package ec.edu.uteq.appwebapi.web;
```

```
import org.springframework.web.bind.annotation.*;
```

```
import java.util.Map;
```

```
@RestController                                // devuelve datos (JSON), no
```

```
vistas
```

```
public class HolaController {
```

```
    @GetMapping("/api/hola")                    // endpoint HTTP GET
```

```
    public Map<String, String> hola(
```

```
        @RequestParam(defaultValue = "Mundo") String nombre) {
```

```
        return Map.of("mensaje", "Hola, " + nombre + "!");
```

```
    }
```

```
}
```

Ejecutar y probar

 **Shift + F10**

Tomcat: puerto 8080

```
GET localhost:8080
/api/hola?nombre=UTEQ
```

200 OK

```
{
  "mensaje":
    "Hola, UTEQ!"
}
```


Arquitectura por capas: los paquetes que creamos

```
ec/edu/uteq/appwebapi/  
├── config/ OpenAPI, CORS  
├── model/ entidades JPA  
│   ├── Carrera.java  
│   ├── Estudiante.java  
│   ├── Perfil.java  
│   ├── Curso.java  
│   ├── Inscripcion.java  
│   └── InscripcionId.java PK doble  
├── repository/  
├── service/  
├── dto/  
├── web/ controllers  
└── exception/
```



web / controller

Recibe HTTP, valida, responde JSON.



service

Lógica de negocio y @Transactional.



repository

Acceso a datos (JpaRepository).



model

Entidades = tablas de la BD.

application.properties: conectar PostgreSQL

```
src/main/resources/application.properties

# --- Origen de datos ---
spring.datasource.url=jdbc:postgresql://localhost:5432/appweb2026ppa
spring.datasource.username=postgres
spring.datasource.password=P@$7Gr3$QL
spring.datasource.driver-class-name=org.postgresql.Driver

# --- JPA / Hibernate ---
spring.jpa.hibernate.ddl-auto=update    # crea/actualiza tablas
spring.jpa.show-sql=true                 # ver el SQL generado
spring.jpa.properties.hibernate.format_sql=true

# --- Swagger UI ---
springdoc.swagger-ui.path=/swagger-ui.html
```

ddl-auto

update

desarrollo

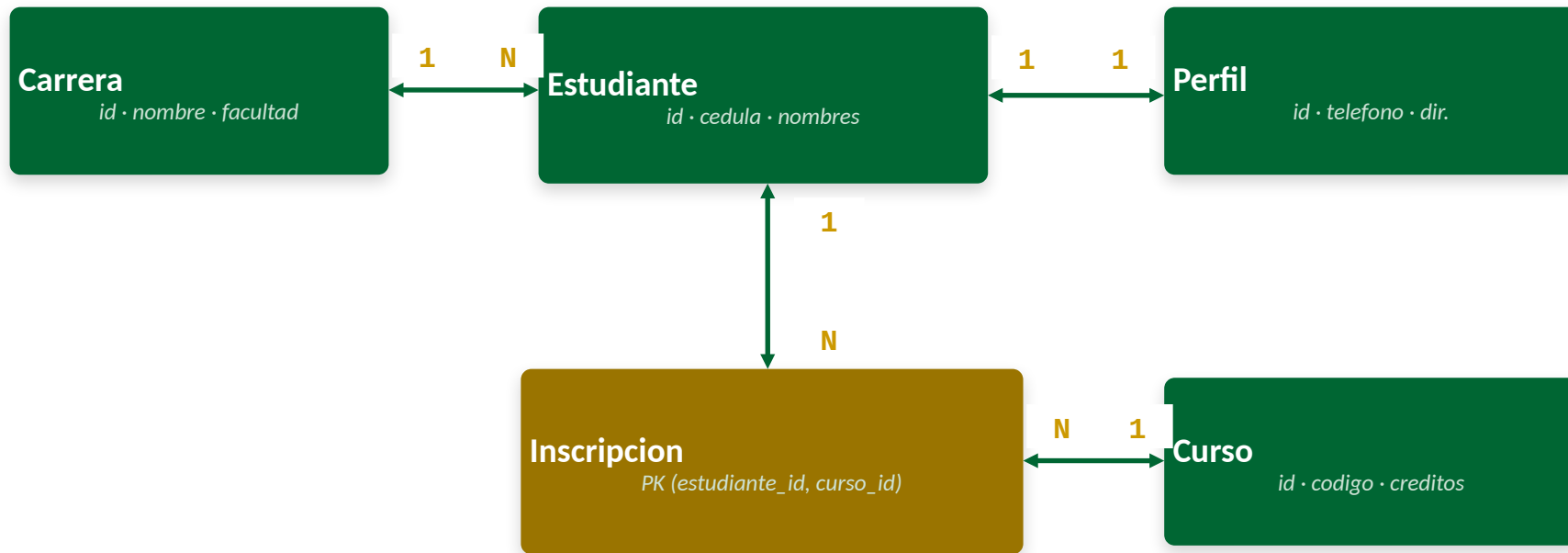
validate

producción

none

Flyway/Liquibase

El diagrama entidad-relación



1:N Carrera→Estudiante 1:1 Estudiante→Perfil N:M Estudiante ↔ Curso (vía Inscripcion)

Estudiante ↔ Perfil: uno a uno

El lado dueño guarda la clave foránea (perfil_id). El inverso usa mappedBy. cascade y orphanRemoval propagan el ciclo de vida.

Perfil.java

```
@Entity @Table(name = "perfil")
@Getter @Setter @NoArgsConstructor
public class Perfil {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String telefono;
    private String direccion;
    private LocalDate fechaNacimiento;

    // lado inverso: la FK vive en Estudiante
    @OneToOne(mappedBy = "perfil")
    private Estudiante estudiante;
}
```

En Estudiante (lado dueño)

```
@OneToOne(cascade =
    CascadeType.ALL,
    orphanRemoval = true)
@JoinColumn(
    name = "perfil_id")
private Perfil perfil;
```

Genera la columna perfil_id (UNIQUE) en la tabla estudiante.

Carrera ↔ Estudiante: uno a muchos

Carrera.java

```
@Entity @Table(name = "carrera")
@Getter @Setter @NoArgsConstructor
public class Carrera {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false, length = 120)
    private String nombre;
    private String facultad;

    // lado inverso (uno): una carrera, muchos estudiantes
    @OneToMany(mappedBy = "carrera")
    private List<Estudiante> estudiantes = new ArrayList<>();
}
```

Regla de oro

El lado «muchos» (N:1) es el dueño y lleva la FK.

El lado «uno» (1:N) usa **mappedBy**.

Usa *FetchType.LAZY* en colecciones para no cargar de más.

Estudiante.java: reúne las tres relaciones

Estudiante.java

```
@Entity @Table(name = "estudiante")
@Getter @Setter @NoArgsConstructor @AllArgsConstructor
public class Estudiante {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    @Column(unique = true, nullable = false, length = 10)
    private String cedula;
    private String nombres;
    private String apellidos;
    private String email;

    @ManyToOne(fetch = FetchType.LAZY)           // N:1 dueño
    @JoinColumn(name = "carrera_id")
    private Carrera carrera;

    @OneToOne(cascade = CascadeType.ALL, orphanRemoval = true)
    @JoinColumn(name = "perfil_id")             // 1:1 dueño
    private Perfil perfil;

    @OneToMany(mappedBy = "estudiante")         // 1:N inverso
    private List<Inscripcion> inscripciones = new ArrayList<>();
}
```

¿Por qué una entidad de asociación?

@ManyToMany simple

Genera una tabla puente automática solo con las dos FK.

✗ No admite columnas extra

(nota, periodo, fecha...).

✗ No puedes referenciar la fila

intermedia como recurso REST.

Entidad de asociación

Promovemos la tabla puente a una **entidad propia: Inscripcion**.

✓ Lleva atributos propios

(nota, periodo, fechaInscripcion).

✓ Su clave primaria es compuesta

por las dos FK: clave de doble campo.

✓ Se modela con @EmbeddedId + @MapsId.

InscripcionId: la clave primaria compuesta

InscripcionId.java

```
@Embeddable // se incrusta en la entidad
@Getter @Setter
@NoArgsConstructor @AllArgsConstructor
public class InscripcionId implements Serializable {

    private Long estudianteId; // primer campo de la clave
    private Long cursoId; // segundo campo de la clave

    // equals() y hashCode() son OBLIGATORIOS en una PK compuesta
    @Override public boolean equals(Object o) {
        if (this == o) return true;
        if (!(o instanceof InscripcionId k)) return false;
        return Objects.equals(estudianteId, k.estudianteId)
            && Objects.equals(cursoId, k.cursoId);
    }
    @Override public int hashCode() {
        return Objects.hash(estudianteId, cursoId);
    }
}
```


Inscripcion.java: @EmbeddedId + @MapsId

Inscripcion.java

```
@Entity @Table(name = "inscripcion")
@Getter @Setter @NoArgsConstructor
public class Inscripcion {

    @EmbeddedId                                // clave primaria compuesta
    private InscripcionId id = new InscripcionId();

    @ManyToOne(fetch = FetchType.LAZY)
    @MapsId("estudianteId")                    // comparte la FK con la PK
    @JoinColumn(name = "estudiante_id")
    private Estudiante estudiante;

    @ManyToOne(fetch = FetchType.LAZY)
    @MapsId("cursoId")
    @JoinColumn(name = "curso_id")
    private Curso curso;

    private BigDecimal nota;                  // atributo propio del enlace
    private String periodo;                   // p.ej. 2026-2027 PI
}
```

@MapsId

Indica que la FK de la relación ES, a la vez, parte de la clave primaria.

Evita columnas duplicadas:
estudiante_id sirve como FK y como mitad de la PK.

Curso.java y el lado inverso de Inscripcion

Curso.java

```
@Entity @Table(name = "curso")
@Getter @Setter @NoArgsConstructor
public class Curso {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    @Column(unique = true, nullable = false, length = 12)
    private String codigo;
    private String nombre;
    private Integer creditos;

    @OneToMany(mappedBy = "curso") // inverso de Inscripcion
    private List<Inscripcion> inscripciones = new ArrayList<>();
}
```

Las 6 clases del modelo

Carrera
Estudiante
Perfil
Curso

Inscripcion
InscripcionId

4 entidades + 1 puente + 1 clave

El SQL (DDL) que Hibernate genera

DDL generado (ddl-auto=update)

```
CREATE TABLE estudiante (  
  id          BIGINT GENERATED ALWAYS AS IDENTITY,  
  cedula      VARCHAR(10) NOT NULL UNIQUE,  
  nombres     VARCHAR(255), apellidos VARCHAR(255),  
  carrera_id  BIGINT REFERENCES carrera (id),  
  perfil_id   BIGINT UNIQUE REFERENCES perfil (id),  
  PRIMARY KEY (id)  
);  
  
CREATE TABLE inscripcion (  
  estudiante_id BIGINT NOT NULL,  
  curso_id      BIGINT NOT NULL,  
  nota          NUMERIC(4,2),  
  periodo       VARCHAR(20),  
  PRIMARY KEY (estudiante_id, curso_id), -- clave de doble campo  
  FOREIGN KEY (estudiante_id) REFERENCES estudiante (id),  
  FOREIGN KEY (curso_id)      REFERENCES curso (id)  
);
```



Clave de doble
campo

PRIMARY KEY
(estudiante_id,
curso_id)

Garantiza que un estudiante no se
inscriba dos veces al mismo curso.

Repositorios: CRUD gratis con Spring Data JPA

```
repository/*.java

public interface EstudianteRepository
    extends JpaRepository<Estudiante, Long> {
    // consultas derivadas: sin escribir SQL
    List<Estudiante> findByCarreraId(Long carreraId);
    Optional<Estudiante> findByCedula(String cedula);
}

public interface CursoRepository
    extends JpaRepository<Curso, Long> {}

// La PK es IncripcionId, no Long:
public interface IncripcionRepository
    extends JpaRepository<Incripcion, IncripcionId> {
    List<Incripcion> findByIdEstudianteId(Long estudianteId);
}
```

Gratis al heredar

```
save()
findById()
findAll()
deleteById()
count()
existsById()
```

El tipo de la PK debe coincidir:
IncripcionId para la tabla puente.

DTOs: separar la entidad de la respuesta JSON

```
dto/EstudianteDTO.java
// record = inmutable y conciso (Java 17+)
public record EstudianteDTO(
    Long id,
    @NotBlank String cedula,
    @NotBlank String nombres,
    @NotBlank String apellidos,
    @Email String email,
    Long carreraId) {}

public record InscripcionDTO(
    Long estudianteId, Long cursoId,
    BigDecimal nota, String periodo) {}
```

¿Por qué DTOs?

Evitan recursión infinita en JSON
(Estudiante ↔ Inscripcion).

Ocultan campos internos y relaciones
perezosas (LAZY).

Validan la entrada con `@NotBlank`,
`@Email`, `@NotNull`.

El servicio: transacciones y mapeo

```
service/EstudianteService.java

@Service
@RequiredArgsConstructor
public class EstudianteService {
    private final EstudianteRepository repo;
    private final CarreraRepository carreraRepo;

    public List<EstudianteDTO> listar() {
        return repo.findAll().stream().map(this::aDTO).toList();
    }

    @Transactional
    public EstudianteDTO crear(EstudianteDTO d) {
        Estudiante e = new Estudiante();
        e.setCedula(d.cedula()); e.setNombres(d.nombres());
        e.setApellidos(d.apellidos()); e.setEmail(d.email());
        e.setCarrera(carreraRepo.getReferenceById(d.carreraId()));
        return aDTO(repo.save(e));
    }
    // aDTO(Estudiante): convierte entidad → DTO ...
}
```

El controlador: GET y POST

web/EstudianteController.java (1/2)

```
@RestController
@RequestMapping("/api/estudiantes")
@RequiredArgsConstructor
public class EstudianteController {
    private final EstudianteService servicio;

    @GetMapping                                // GET /api/estudiantes
    public List<EstudianteDTO> listar() { return servicio.listar(); }

    @GetMapping("/{id}")                      // GET /api/estudiantes/{id}
    public EstudianteDTO uno(@PathVariable Long id) {
        return servicio.buscar(id);
    }

    @PostMapping                              // POST → 201 Created
    @ResponseStatus(HttpStatus.CREATED)
    public EstudianteDTO crear(@Valid @RequestBody EstudianteDTO d) {
        return servicio.crear(d);
    }
}
```

PUT, DELETE y la clave compuesta en la URL

EstudianteController.java (2/2)

```
@PutMapping("/{id}") // reemplaza por completo
public EstudianteDTO actualizar(@PathVariable Long id,
    @Valid @RequestBody EstudianteDTO d) {
    return servicio.actualizar(id, d);
}

@DeleteMapping("/{id}") // 204 No Content
@ResponseStatus(HttpStatus.NO_CONTENT)
public void eliminar(@PathVariable Long id) {
    servicio.eliminar(id);
}
```

Verbo → respuesta

POST	→ 201 Created
GET	→ 200 OK
PUT	→ 200 OK
DELETE	→ 204 No Content
404	si no existe

InscripcionController.java

```
// La PK compuesta se direcciona con dos variables en la URL:
@PostMapping("/api/inscripciones") // crea inscripcion
public InscripcionDTO inscribir(@RequestBody InscripcionDTO d){ ... }

@GetMapping("/api/estudiantes/{id}/cursos") // navega la relacion
public List<CursoDTO> cursosDe(@PathVariable Long id){ ... }
```


Manejo de errores y validación

exception/ManejadorErrores.java

```
@RestControllerAdvice           // captura excepciones globalmente
public class ManejadorErrores {

    @ExceptionHandler(RecursoNoEncontrado.class)
    @ResponseStatus(HttpStatus.NOT_FOUND)    // 404
    public Map<String,String> noEncontrado(RecursoNoEncontrado e){
        return Map.of("error", e.getMessage());
    }

    @ExceptionHandler(MethodArgumentNotValidException.class)
    @ResponseStatus(HttpStatus.BAD_REQUEST)    // 400
    public Map<String,String> validacion(
        MethodArgumentNotValidException e) {
        var err = new HashMap<String,String>();
        e.getBindingResult().getFieldErrors().forEach(f ->
            err.put(f.getField(), f.getDefaultMessage()));
        return err;
    }
}
```

Mapa de verbos HTTP y un ejemplo con cURL

Verbo	URI	Acción	Código
GET	/api/estudiantes	Listar	200
GET	/api/estudiantes/7	Obtener uno	200 · 404
POST	/api/estudiantes	Crear	201
PUT	/api/estudiantes/7	Reemplazar	200
DELETE	/api/estudiantes/7	Eliminar	204
POST	/api/inscripciones	Inscribir	201

Terminal

```
$ curl -X POST \
  localhost:8080/api/
estudiantes \
  -H 'Content-Type:
  application/json' \
  -d '{"cedula":"120...",
  "nombres":"Ana"}'

< 201 Created
< Location: /api/
estudiantes/7
```

Swagger UI + checklist profesional



OpenAPI 3 + Swagger UI

springdoc-openapi 3.0.3 genera la documentación automáticamente.

Sin configurar nada, abre:

`localhost:8080/swagger-ui.html`

`localhost:8080/v3/api-docs`

Anota con `@Operation` y `@Schema` para enriquecer la API.

Checklist de la semana

- ✓ Capas separadas: controller → service → repository
- ✓ DTOs distintos de las entidades JPA
- ✓ `@Valid` en toda entrada del controlador
- ✓ Relaciones LAZY + cascade donde corresponda
- ✓ PK compuesta con `@EmbeddedId` + `@MapsId`
- ✓ Verbos y códigos HTTP correctos (201, 204, 404)
- ✓ API documentada con OpenAPI / Swagger



C I E R R E

De un endpoint a una API completa

Construiste un servicio REST con Spring Boot 4: proyecto, capas, entidades con las cuatro cardinalidades y una **clave primaria de doble campo** resuelta con @EmbeddedId + @MapsId.

Lo que sigue: una SPA en Angular consumirá esta misma API REST sin tocar el backend.

¡Gracias!